Kubernetes ConfigMap & Environment Variables (ENV)

This README explains how to use **ConfigMap** with **Environment Variables (ENV)** in Kubernetes and the different ways to consume them inside a container.

What is a ConfigMap?

A **ConfigMap** in Kubernetes is an object used to store **configuration data** for applications separately from the container image.

This helps you:

- Keep configuration outside the application code
- Reuse configuration across multiple pods
- Update configuration without rebuilding images

Example ConfigMap

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_COLOR: blue
  APP_MODE: production
  APP_PORT: "8080"
```

This ConfigMap stores three configuration values:

- `APP_COLOR`
- `APP_MODE`
- `APP_PORT`

Ways to Use ConfigMap in a Pod

There are **three main ways** to use a ConfigMap inside a container.

1. envFrom (Load all values as environment variables)
2. env (Load a single value as an environment variable)
3. Volume (Mount ConfigMap as files)

1. ENV From ConfigMap (envFrom)

This method loads **all keys from the ConfigMap** as environment variables.

Example

```
apiVersion: v1
kind: Pod
metadata:
  name: envfrom-pod
spec:
  containers:
    - name: app-container
      image: nginx
      envFrom:
        - configMapRef:
            name: app-config
```

Result inside the container

The container will have:

```
APP_COLOR=blue
APP_MODE=production
APP_PORT=8080
```

When to use envFrom

Use it when:

- You want **all variables from the ConfigMap**
- Your application uses many environment variables
- You want simpler configuration

2. Single ENV Variable (env)

This method loads **a specific key from the ConfigMap**.

Example

```
apiVersion: v1
kind: Pod
```

```
  metadata:
    name: single-env-pod
  spec:
    containers:
      - name: app-container
        image: nginx
        env:
          - name: APP_COLOR
            valueFrom:
              configMapKeyRef:
                name: app-config
                key: APP_COLOR
```

Result inside the container

```
APP_COLOR=blue
```

When to use env

Use it when:

- You only need **specific variables**
- You want to control variable names
- You want to avoid loading unnecessary config

Example:

```
- name: COLOR
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: APP_COLOR
```

This creates:

```
COLOR=blue
```

---

3. ConfigMap as Volume

Instead of environment variables, you can mount the ConfigMap as **files inside the container**.

Each key becomes a file.

Example

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-volume-pod
spec:
  containers:
    - name: app-container
      image: nginx
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: app-config
```

Result inside container

Files created:

```
/etc/config/APP_COLOR
/etc/config/APP_MODE
/etc/config/APP_PORT
```

Example:

```
cat /etc/config/APP_COLOR
blue
```

When to use Volume

Use it when:

- Your application reads configuration from **files**
- You need config formats like:
  - `.conf`
  - `.yaml`
  - `.json`
  - `.properties`

Example use cases:

- Nginx configuration
- Spring Boot config

- Application YAML files

Comparison

Method	What it does	Use Case
envFrom	Loads all ConfigMap keys as ENV variables	Simple apps with many variables
env	Loads a specific key from ConfigMap	When only one variable is needed
Volume	Mounts ConfigMap as files	Apps that read configuration files

---

Best Practices

- Use **ConfigMaps for non-sensitive configuration**
- Use **Secrets for passwords, tokens, or credentials**
- Avoid hardcoding configuration inside container images
- Use `envFrom` for large configurations
- Use `Volume` for applications that expect config files

Summary

ConfigMaps allow you to separate configuration from application code.
They can be consumed by containers in three main ways:

1. **envFrom** → load all values as environment variables
2. **env** → load specific values as environment variables
3. **volume** → mount configuration as files

This makes applications easier to manage, configure, and deploy in Kubernetes.